

就像所有用上 `static` 对象的设计一样, 这一个也立刻造成我们对多线程安全性的疑虑。不过那还只是它显而易见的弱点。如果想看看更深层的瑕疵, 考虑以下面这些完全合理的客户代码:

```
bool operator==(const Rational& lhs,          //一个针对 Rationals
                const Rational& rhs); // 而写的 operator==
Rational a, b, c, d;
...
if ((a * b) == (c * d)) {
    当乘积相等时, 做适当的相应动作;
} else {
    当乘积不等时, 做适当的相应动作;
}
```

猜想怎么着? 表达式 `((a*b) == (c*d))` 总是被核算为 `true`, 不论 `a, b, c` 和 `d` 的值是什么!

一旦将代码重新写为等价的函数形式, 很容易就可以了解出了什么意外:

```
if (operator==(operator*(a, b), operator*(c, d)))
```

注意, 在 `operator==` 被调用前, 已有两个 `operator*` 调用式起作用, 每一个都返回 `reference` 指向 `operator*` 内部定义的 `static Rational` 对象。因此 `operator==` 被要求将 “`operator*` 内的 `static Rational` 对象值” 拿来和 “`operator*` 内的 `static Rational` 对象值” 比较, 如果比较结果不相等, 那才奇怪呢。(译注: 这里我补充说明: 两次 `operator*` 调用的确各自改变了 `static Rational` 对象值, 但由于它们返回的都是 `reference`, 因此调用端看到的永远是 `static Rational` 对象的 “现值”。)

这应该足够说服你, 欲令诸如 `operator*` 这样的函数返回 `reference`, 只是浪费时间而已, 但现在或许又有些人这样想: “唔, 如果一个 `static` 不够, 或许一个 `static array` 可以得分……”。

我不打算再次写出示例来驳斥这个想法以彰显自己多么厉害, 但我可以简单描述为什么你该为了提出这个念头而脸红。首先你必须选择 `array` 大小 `n`。如果 `n` 太小, 你可能会耗尽 “用以存储函数返回值” 的空间, 那么情况就回到了我们刚才讨论过的单一 `static` 设计。但如果 `n` 太大, 会因此降低程序效率, 因为 `array` 内的每一个对象都会在函数第一次被调用时构造完成。那么将消耗 `n` 个构造函数和 `n` 个析构函数——即使我们所讨论的函数只被调用一次。如果所谓 “最优化” 是改善软件效率的过程, 我们现在所谈的这些应该称为 “恶劣化”。最后, 想一想如何将你需要的值放进 `array` 内,

而那么做的成本又是多少。在对象之间搬移数值的最直接办法是通过赋值 (*assignment*) 操作, 但赋值的成本几何? 对许多 *types* 而言它相当于调用一个析构函数 (用以销毁旧值) 加上一个构造函数 (用以复制新值)。但你的目标是避免构造和析构成本耶! 面对现实吧, 这个做法不会成功的。就算以 *vector* 替换 *array* 也不会让情况更好些。

一个“必须返回新对象”的函数的正确写法是: 就让那个函数返回一个新对象呗。对 *Rational* 的 *operator** 而言意味以下写法 (或其他本质上等价的代码):

```
inline const Rational operator * (const Rational& lhs, const Rational& rhs)
{
    return Rational(lhs.n * rhs.n, lhs.d * rhs.d);
}
```

当然, 你得承受 *operator** 返回值的构造成本和析构成本, 然而长远来看那只是为了获得正确行为而付出的一个小小代价。但万一账单很恐怖, 你承受不起, 别忘了 C++ 和所有编程语言一样, 允许编译器实现者施行最优化, 用以改善产出码的效率却不改变其可观察的行为。因此某些情况下 *operator** 返回值的构造和析构可被安全地消除。如果编译器运用这一事实 (它们也往往如此), 你的程序将继续保持它们该有的行为, 而执行起来又比预期的更快。

我把以上的讨论浓缩总结为: 当你必须在“返回一个 *reference* 和返回一个 *object*”之间抉择时, 你的工作就是挑出行为正确的那个。就让编译器厂商为“尽可能降低成本”鞠躬尽瘁吧, 你可以享受你的生活。

请记住

- 绝不要返回 *pointer* 或 *reference* 指向一个 *local stack* 对象, 或返回 *reference* 指向一个 *heap-allocated* 对象, 或返回 *pointer* 或 *reference* 指向一个 *local static* 对象而有可能同时需要多个这样的对象。条款 4 已经为“在单线程环境中合理返回 *reference* 指向一个 *local static* 对象”提供了一份设计实例。

条款 22: 将成员变量声明为 *private*

Declare data members *private*.

OK, 下面是我的规划。首先带你看看为什么成员变量不该是 *public*, 然后让你看看所有反对 *public* 成员变量的论点同样适用于 *protected* 成员变量。最后导出一个结论: 成员变量应该是 *private*。获得这个结论后, 本条款也就大功告成了。

好, 现在看看 `public` 成员变量。为什么不采用它呢?

让我们从语法一致性开始 (同时请见条款 18)。如果成员变量不是 `public`, 客户唯一能够访问对象的办法就是通过成员函数。如果 `public` 接口内的每样东西都是函数, 客户就不需要在打算访问 `class` 成员时迷惑地试着记住是否该使用小括号 (圆括号)。他们只要做就是了, 因为每样东西都是函数。就生命而言, 这至少可以省下许多搔首弄耳的时间。

或许你不认为一致性的理由足以令人信服, 那么这个事实如何: 使用函数可以让你对成员变量的处理有更精确的控制。如果你令成员变量为 `public`, 每个人都可以读写它, 但如果你以函数取得或设定其值, 你就可以实现出“不准访问”、“只读访问”以及“读写访问”。见鬼了, 你甚至可以实现“惟写访问”, 如果你想要的话:

```
class AccessLevels {
public:
    ...
    int getReadOnly() const { return readOnly; }
    void setReadWrite(int value) { readWrite = value; }
    int getReadWrite() const { return readWrite; }
    void setWriteOnly(int value) { writeOnly = value; }
private:
    int noAccess;           //对此 int 无任何访问动作
    int readOnly;           //对此 int 做只读访问 (read-only access)
    int readWrite;          //对此 int 做读写访问 (read-write access)
    int writeOnly;          //对此 int 做惟写访问 (write-only access)
};
```

如此细微地划分访问控制颇有必要, 因为许多成员变量应该被隐藏起来。每个成员变量都需要一个 `getter` 函数和 `setter` 函数毕竟罕见。

还是不够说服你? 是端出大口径武器的时候了: 封装啦。如果你通过函数访问成员变量, 日后可改以某个计算替换这个成员变量, 而 `class` 客户一点也不会知道 `class` 的内部实现已经起了变化。